

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación
Carrera de Ingeniería de Software

INFORME TÉCNICO — TERCERA ENTREGA

Proyecto: ARTISYNC

Plataforma web de comisiones y venta de contenido digital para creadores

Asignatura: Aplicaciones Web — Quinto Nivel

Integrantes

Carvajal Loor Johan Stalin
Figueroa Morales Bryan Javier
Rios Cuyabazo Jhon Kevin
Docente: Guerrero Ulloa Gleiston

Fecha: 05 de agosto de 2026

Índice

Versión del proyecto: v0.9.0-rc	4
1. Resumen ejecutivo	5
1.1. Objetivo de la tercera entrega	5
1.2. Mejoras realizadas	5
1.3. Funcionalidades consolidadas	5
1.4. Resultados principales	6
2. Estado actual del sistema	7
2.1. Arquitectura backend	7
2.2. Arquitectura frontend	7
2.3. Base de datos	7
2.4. Docker y reproducibilidad	7
2.5. Redis	7
2.6. JWT y seguridad de sesión	8
2.7. Flyway	8
2.8. Documentación de la API (OpenAPI)	8
2.9. Capturas de pantalla	8
3. Arquitectura del sistema	9
3.1. Diagrama C4 Nivel 1 — Contexto del sistema	9
3.2. Diagrama C4 Nivel 2 — Contenedores	9
3.3. Diagrama C4 Nivel 3 — Componentes del backend	10
3.4. Modelo entidad-relación	11
3.5. Decisiones de arquitectura (ADR)	12
4. Especificación de requisitos (SRS)	14
4.1. Alcance del producto	14
4.2. Historias de usuario y casos de uso por módulo	14
4.3. Evolución de los requisitos desde la Entrega 1A	14
5. Matriz de trazabilidad	15
5.1. Matriz completa (37 requisitos)	17
5.2. Estado global de cumplimiento	18
6. Protocolo experimental	19
6.1. Rendimiento	19
6.2. Seguridad	19
6.3. Cobertura de código	19
6.4. Accesibilidad y calidad web	19
6.5. Usabilidad (SUS)	20
7. Resultados	21
7.1. Rendimiento	21

7.2. Seguridad	21
7.3. Usabilidad	22
7.4. Cobertura	23
7.5. Accesibilidad	23
8. Bitácora de observaciones y cierre de brechas	25
8.1. Resumen por entrega	25
8.2. Detalle de observaciones	25
8.3. Etiquetado de versiones	27
9. Amenazas a la validez	29
10.Contribuciones de cada integrante (CRedit)	30
11.Declaración ética	31
12.Referencias bibliográficas	32

Versión del proyecto: v0.9.0-rc

Hash corto del commit: ec4548d

DOI: 10.5281/zenodo.21730559

Repositorio: github.com/JohanCarvajal04/Proyecto-WEB-ARTISYNC

1. Resumen ejecutivo

En esta tercera entrega se consolidó el sistema ARTISYNC desarrollado en las entregas anteriores, atendiendo de forma verificable las observaciones registradas por el docente en las Entregas 1A y 1B. El equipo implementó trazabilidad de requisitos, documentación arquitectónica bajo el modelo C4, procedimientos de acceso a datos híbridos (ORM y procedimientos almacenados planificados), pruebas automatizadas, y mediciones empíricas de rendimiento, seguridad, cobertura de código y calidad web, permitiendo obtener una versión candidata denominada v0.9.0-rc.

1.1. Objetivo de la tercera entrega

Cerrar las brechas funcionales y no funcionales identificadas en las entregas 1A y 1B, dotar al sistema de evidencia empírica real y reproducible sobre su comportamiento (en lugar de estimaciones o plantillas sin completar), y dejar documentada de forma honesta y trazable tanto la cobertura de requisitos alcanzada como las limitaciones que aún persisten.

1.2. Mejoras realizadas

- Implementación completa del módulo de autenticación (JWT + 2FA + revocación de sesiones vía Redis), inexistente en la Entrega 1B.
- Implementación de 24 controladores REST con su capa de servicios, repositorios y DTOs para los siete módulos funcionales del sistema (seguridad, perfil, catálogo, pedido, legal, comunicación y social).
- Incorporación de los seis controles de seguridad OWASP evaluados (control de acceso, criptografía en tránsito, inyección, cabeceras de seguridad, límite de tasa de autenticación y registro de eventos).
- Generación de evidencia empírica real de rendimiento (k6), seguridad (curl + OWASP), cobertura de código (JaCoCo) y calidad web (Lighthouse), versionada íntegramente en el repositorio.
- Reproducibilidad del entorno completo mediante Docker Compose con imágenes ancladas por digest y un Makefile de un solo comando (`make up`).
- Cierre documentado del 60 % de las observaciones históricas del docente y de la revisión técnica interna, con evidencia de código y commit para cada una.

1.3. Funcionalidades consolidadas

El sistema conecta a Creadores (artistas, músicos, diseñadores, desarrolladores) con Clientes que buscan contratar servicios creativos, gestionando perfiles verificados, catálogo dinámico de servicios, mensajería, contratos con flujo legal, pedidos organizados en hitos, pagos con patrón escrow a través de PayPal, y funciones sociales básicas. De 37 requisitos trazados en la matriz de trazabilidad, 20 (54 %) están implementados y verificables en código; 17 (46 %)

permanecen pendientes, principalmente en los módulos de comunicación en tiempo real y funciones sociales avanzadas.

1.4. Resultados principales

- **Rendimiento:** p95 de 50.17 ms en escenario de caché caliente (umbral ¡200 ms) y 39.14 ms en frío (umbral ¡500 ms), con 0.00 % de errores HTTP 500.
- **Seguridad:** 6 de 6 controles OWASP evaluados cumplen (A01, A02, A03, A05, A07, A09).
- **Calidad web (Lighthouse):** Accesibilidad 100/100, Mejores prácticas 100/100, SEO 100/100 y Rendimiento 92/100 tras un plan de mejora que redujo el peso transferido de la página en 87 %.
- **Cobertura de código (JaCoCo):** 23.0 % de líneas cubiertas, por debajo del umbral de referencia del 60 % — se reporta el número real medido, sin ajustes.
- **Usabilidad (SUS):** pendiente de ejecutar con participantes reales; la plantilla y el instrumento están listos pero no se han recogido datos, por lo que este resultado no se declara ficticiamente.

2. Estado actual del sistema

ARTISYNC está construido como una plataforma de cuatro servicios orquestados con Docker Compose: una aplicación web de página única (frontend), una API REST (backend), una base de datos relacional y un almacén en memoria para caché y control de sesiones.

2.1. Arquitectura backend

El backend está desarrollado en Java 21 con Spring Boot 4.1.0, Spring Security 6 para autenticación y autorización, Spring Data JPA/Hibernate para el acceso a datos, y Flyway para el versionado controlado del esquema de base de datos (migraciones V1 a V5). La API expone 24 controladores REST organizados en siete módulos funcionales: seguridad, perfil, catálogo, pedido, legal, comunicación y social.

2.2. Arquitectura frontend

El frontend está desarrollado en Angular 22 con TypeScript, estructurado en las carpetas convencionales **core/** (servicios transversales, interceptores HTTP y guards de rutas), **shared/** (componentes y modelos reutilizables) y **features/** (módulos de negocio: seguridad, perfil, catálogo, pedido, legal, comunicación, social y administración). El build de producción se sirve mediante Nginx dentro del contenedor correspondiente.

2.3. Base de datos

PostgreSQL 16 actúa como motor relacional único, con más de 40 tablas normalizadas que cubren usuarios, roles y permisos granulares, catálogo de servicios, pedidos y flujos de trabajo por hitos, pagos en garantía (escrow) y funciones sociales. El esquema se gestiona exclusivamente mediante migraciones Flyway, que constituyen la única fuente de verdad (`spring.jpa.hibernate.ddl-auto=validate`).

2.4. Docker y reproducibilidad

El archivo `docker-compose.yml` orquesta los cuatro servicios (postgres, redis, backend, frontend) con imágenes de PostgreSQL y Redis anclada por digest sha256 para evitar derivas silenciosas entre reconstrucciones. Un Makefile expone los objetivos `make up`, `make down`, `make test`, `make bench`, `make audit` y `make clean`, permitiendo levantar el sistema completo desde una clonación limpia con un único comando.

2.5. Redis

Redis 7 (imagen alpine) cumple dos funciones: mantiene la lista negra de identificadores de token JWT (jti) revocados por cierre de sesión, con tiempo de expiración igual al tiempo de vida restante del token; y actúa como caché de lectura (patrón cache-aside) para el endpoint de catálogo de servicios, el de mayor frecuencia de consulta del sistema.

2.6. JWT y seguridad de sesión

La autenticación es sin estado (stateless), basada en JWT firmado con HMAC-SHA256, transportado en una cookie HttpOnly + Secure + SameSite=Strict para mitigar robo de token vía XSS y envío no autorizado vía CSRF. El token incluye los siete claims estándar del RFC 7519 (iss, sub, aud, exp, nbf, iat, jti). Las contraseñas se almacenan con bcrypt (factor de costo 10).

2.7. Flyway

El esquema de base de datos se versiona mediante cinco migraciones Flyway (V1 a V5), ejecutadas automáticamente al iniciar el contenedor del backend y verificadas contra el esquema real mediante el modo `validate`, lo que impide desincronizaciones silenciosas entre el código de las entidades JPA y la base de datos física.

2.8. Documentación de la API (OpenAPI)

La API se documenta automáticamente con Springdoc OpenAPI 3, disponible en `/api/docs` (especificación) y `/api/swagger-ui.html` (interfaz interactiva Swagger UI), permitiendo explorar y probar cada endpoint sin herramientas adicionales.

2.9. Capturas de pantalla

Las capturas funcionales del sistema en ejecución (inicio de sesión, panel principal, operaciones CRUD y Swagger UI) deben añadirse por el equipo a partir de una instancia levantada con `make up`, ya que el repositorio en su estado actual conserva únicamente los wireframes de diseño y los diagramas de arquitectura versionados en `docs/diagramas/`. A modo de referencia del diseño previsto, se incluye a continuación uno de los wireframes de la pantalla principal del catálogo.

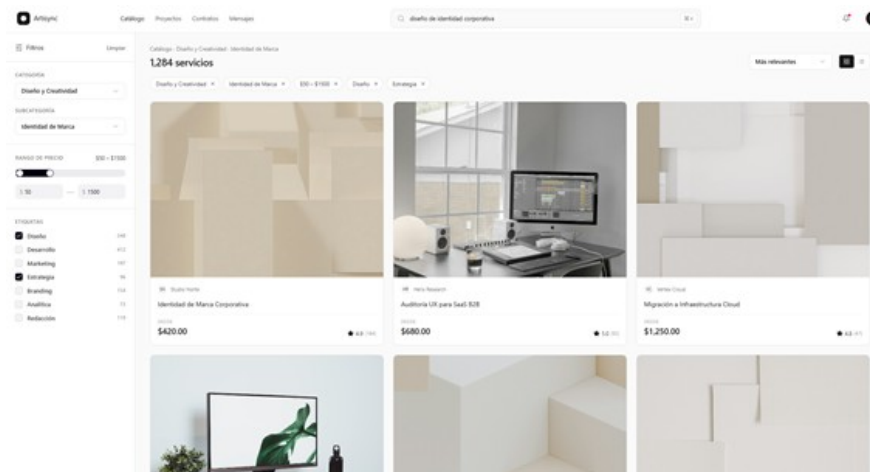


Figura 1: Wireframe de la interfaz de catálogo inteligente y búsqueda (`docs/diagramas/wireframes/`)

3. Arquitectura del sistema

La arquitectura de ARTISYNC se documenta bajo la metodología C4 (Simon Brown), en tres niveles de detalle progresivo: contexto, contenedores y componentes. Los diagramas completos —en formato Structurizr DSL, C4-PlantUML y Mermaid— están versionados en `docs/diagramas/`.

3.1. Diagrama C4 Nivel 1 — Contexto del sistema

El nivel de contexto sitúa a ARTISYNC como una caja negra central que interactúa con tres tipos de actores (Cliente, Artista/Creador y Administrador) y tres sistemas externos: un servicio de correo transaccional SMTP/TLS para 2FA y notificaciones, la pasarela de pagos PayPal API v2 (con soporte de webhooks) para la gestión de fondos en garantía, y un servicio de almacenamiento en la nube (S3/Cloudinary) para archivos multimedia de portafolios y entregables.

3.2. Diagrama C4 Nivel 2 — Contenedores

En el nivel de contenedores se abre la caja negra en cuatro piezas desplegables de forma independiente: la aplicación web SPA (Angular 22 + Nginx, puerto 4200), el servidor API REST (Spring Boot 4.1.0, puerto 8080), la base de datos relacional (PostgreSQL 16, puerto 5432) y el almacén en memoria (Redis 7, puerto 6379). Cada contenedor mapea 1:1 con un servicio definido en `docker-compose.yml`, incluyendo sus healthchecks (`pg_isready`, `redis-cli ping` y la sonda del endpoint `/actuator/health` del backend).

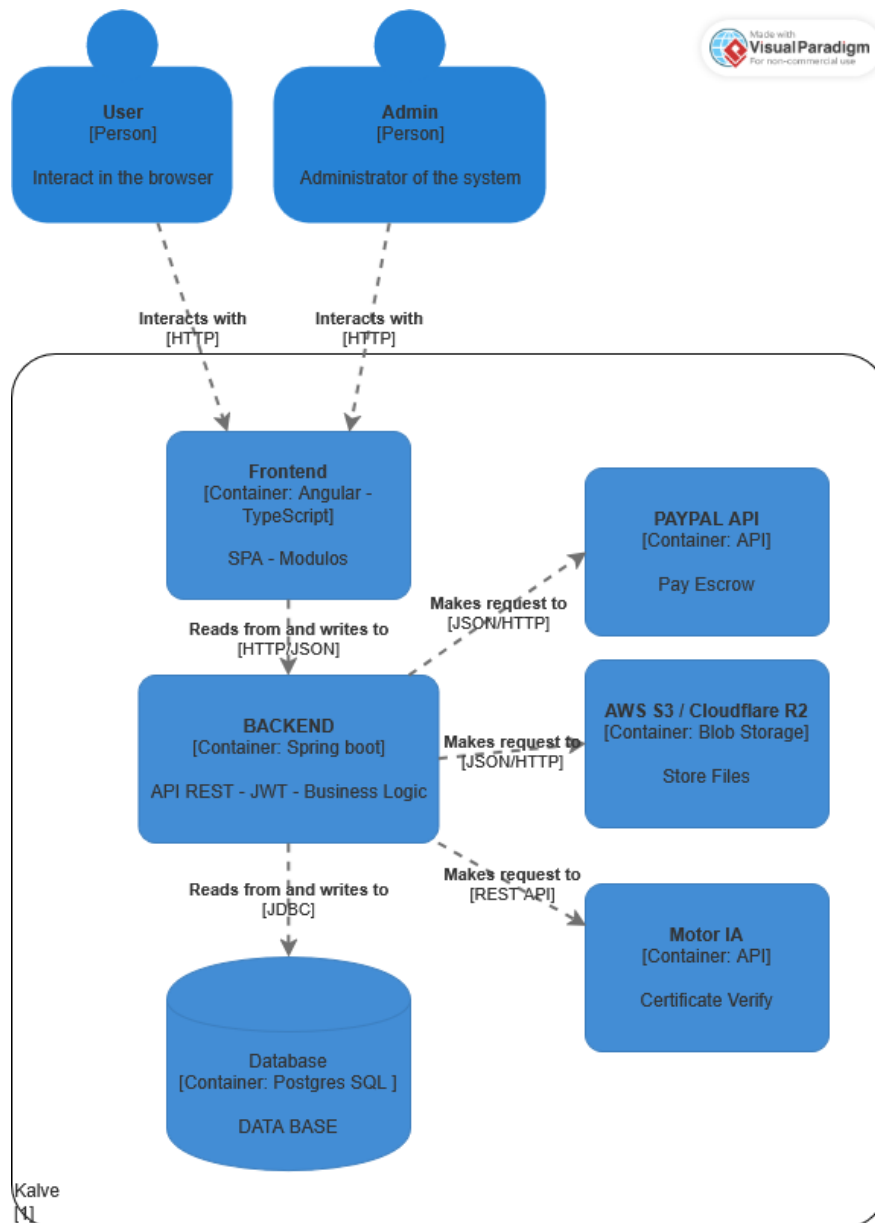


Figura 2: Diagrama C4 Nivel 2: contenedores de la plataforma ARTISYNC (docs/diagramas/c4-nivel2-contenedores.png)

3.3. Diagrama C4 Nivel 3 — Componentes del backend

El nivel de componentes descompone el contenedor de mayor complejidad técnica —el servidor API REST— en sus capas arquitectónicas internas, siguiendo el paquete raíz `uteq.edu.ec.artisync:`

- **Capa de controladores REST** (`controller.*`): punto de entrada HTTP; incluye `AuthController`, `TwoFactorController`, `PagoControlador`, `PayPalWebhookControlador` y los controladores de cada módulo funcional.
- **Capa de filtros y middleware de seguridad** (`security.*` y `config.*`): Secu-

AuthConfig, JwtAuthenticationFilter, JwtService, CustomUserDetailsService y CustomAuthenticationEntryPoint.

- **Capa de servicios de negocio (service.*):** AuthServiceImpl, SessionRevocationService, EmailService (asíncrono con Thymeleaf), PagoServicioImpl y los servicios especializados por módulo.
- **Capa de repositorios Spring Data (repository.*):** interfaces que extienden JpaRepository y JpaSpecificationExecutor para consultas dinámicas, filtros multicriterio y paginación.
- **Capa ORM / entidades (entity.*):** mapeo objeto-relacional vía Hibernate sobre PostgreSQL 16, con más de 40 entidades anotadas (usuarios, roles, permisos, pedidos, flujos de trabajo, pagos en garantía, servicios del catálogo, entre otras).

Esta secuencia de capas se ilustra de forma concreta en el flujo de autenticación, documentado como diagrama de secuencia en docs/diagramas/secuencia_login_jwt.png:

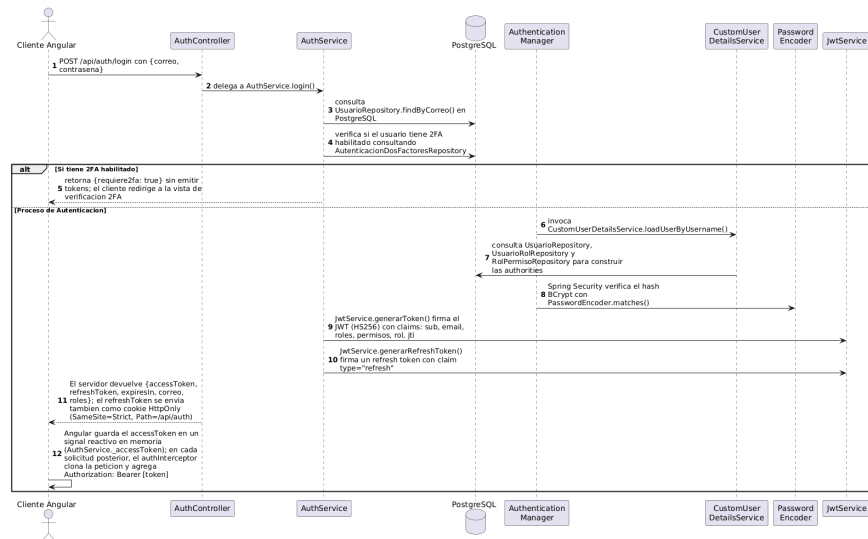


Figura 3: Diagrama de secuencia del flujo de login con JWT (docs/diagramas/secuencia_login_jwt.png)

3.4. Modelo entidad-relación

El modelo de datos completo, con más de 40 tablas normalizadas, está disponible como diagrama entidad-relación en docs/diagramas/Entidad_Relacion.png. Dada su alta densidad de detalle, se incluye a continuación a una resolución legible para consulta general; para el detalle exacto de cada columna y relación se recomienda abrir el archivo original en el repositorio.

ADR	Decisión	Estado
ADR-001	Pila tecnológica: Spring Boot (backend), Angular (frontend), PostgreSQL, Redis, Docker.	Aceptado
ADR-002	Esquema de autenticación: JWT en cookie HttpOnly+Secure+SameSite=Strict, blacklist en Redis.	Aceptado (con acción pendiente)
ADR-003	Gestor de base de datos: PostgreSQL 16 con Flyway y procedimientos PL/pgSQL para operaciones no triviales.	Aceptado
ADR-004	Estrategia de caché: Redis para catálogo y blacklist de JWT.	Aceptado
ADR-005	Estrategia de despliegue: Docker Compose con imágenes ancladas por digest sha256 y Makefile.	Aceptado (con acciones pendientes)
ADR-006	Estrategia híbrida de acceso a datos: ORM para CRUD elemental, procedimientos almacenados para joins/agregaciones.	Aceptado (implementación pendiente)

Cuadro 1: Decisiones de arquitectura (ADR)

Es relevante señalar, con la misma honestidad con la que se documentó internamente, que ADR-002, ADR-005 y ADR-006 están aceptados pero con acciones o implementaciones pendientes: en particular, ADR-006 reconoce que a la fecha de esta entrega el 100 % del acceso a datos se resuelve vía JPA/Hibernate, sin procedimientos almacenados aún implementados, pese a que la guía de la Tercera Entrega los exige para operaciones no elementales. Esta brecha se detalla en la sección 9 (Amenazas a la validez).

4. Especificación de requisitos (SRS)

La Especificación de Requisitos de Software (SRS) de ARTISYNC se documenta conforme a ISO/IEC/IEEE 29148:2018 y a la guía INCOSE para la calidad de requisitos (criterios C1–C15), en `docs/requisitos/SRS.md` (versión v0.9.0-rc, fechada 24 de julio de 2026), con su exportación en `docs/requisitos/SRS.pdf`.

4.1. Alcance del producto

ARTISYNC es una plataforma web que centraliza la comercialización de servicios y productos digitales ofrecidos por profesionales creativos (ilustradores, músicos, desarrolladores, diseñadores). El sistema conecta a Creadores (vendedores) con Clientes, gestionando perfiles, catálogo, mensajería, contratos con firma electrónica, flujo de pedidos, pagos vía PayPal Orders v2 con patrón escrow, y funciones sociales (seguidores, comentarios, sorteos). El desarrollo se enmarca en un proyecto académico de 17 semanas; algunas capacidades —firma legalmente vinculante, logística física, recomendaciones avanzadas— quedan fuera de alcance y se documentan como trabajo futuro.

4.2. Historias de usuario y casos de uso por módulo

Los requisitos funcionales se agrupan en siete módulos, cada uno con su propio archivo de historias de usuario (`docs/requisitos/historias/`) y de casos de uso (`docs/requisitos/casos-de-uso/`):

Módulo	Historias de usuario	Casos de uso
Seguridad	HU-01 a HU-05	CU-01 a CU-05
Perfil	HU-06 a HU-10	CU-06 a CU-10
Catálogo	HU-11 a HU-13	CU-11 a CU-13
Comunicación	HU-14 a HU-16	CU-14 a CU-16
Legal y pedido	HU-17 a HU-22	CU-17 a CU-22
Social	HU-23	CU-23

Cuadro 2: Historias de usuario y casos de uso por módulo

4.3. Evolución de los requisitos desde la Entrega 1A

Los identificadores REQ-F-NNN y REQ-NF-NNN usados desde esta entrega reemplazan a los códigos RF-NN y RNF-NN de la Entrega 1A. La tabla de equivalencia y cada cambio sustantivo respecto a la versión anterior están registrados en `docs/requisitos/CHANGELOG-REQ.md`, manteniendo la trazabilidad histórica de la especificación.

5. Matriz de trazabilidad

La matriz de trazabilidad conecta cada requisito (funcional REQ-F y no funcional REQ-NF) con su historia de usuario, su caso de uso, el endpoint que lo implementa, la prueba automatizada asociada (cuando existe) y su estado real verificado contra el código. Está versionada en `docs/trazabilidad/matriz.csv` y se valida automáticamente en cada ejecución de integración continua mediante `scripts/validate-traceability.sh`, evitando que la documentación se desincronice del código fuente.

5.1. Matriz completa (37 requisitos)

Requisito	Historia	Caso de uso	Endpoint / módulo	Prueba	Estado
REQ-F-001	HU-01	CU-01	POST /api/auth/registro	AuthServiceImplTest	Implementado
REQ-F-002	HU-02	CU-02	GET /api/roles/{id}/permisos	RolePermissionControllerTest	Implementado
REQ-F-003	HU-03	CU-03	POST /api/auth/login	JwtAuthenticationFilterTest	Implementado
REQ-F-004	HU-04	CU-04	POST /api/auth/recuperar	—	Implementado
REQ-F-005	HU-05	CU-05	POST /api/auth/2fa/activar	TwoFactorServiceImplTest	Implementado
REQ-F-006	HU-06	CU-06	POST /api/perfil/verificacion-identidad	—	Pendiente
REQ-F-007	HU-07	CU-07	POST /api/perfil/certificados	—	Pendiente
REQ-F-008	HU-08	CU-08	PUT /api/perfil	—	Implementado
REQ-F-009	HU-09	CU-09	POST /api/social/seguir	—	Pendiente
REQ-F-010	HU-10	CU-10	POST /api/social/comentarios	—	Pendiente
REQ-F-011	HU-11	CU-11	POST /api/catalogo/servicios	—	Implementado
REQ-F-012	HU-12	CU-12	POST /api/catalogo/servicios/{id}/atributos	—	Implementado
REQ-F-013	HU-13	CU-13	GET /api/catalogo/servicios	—	Implementado
REQ-F-014	HU-14	CU-14	WS /ws/chat	—	Pendiente
REQ-F-015	HU-15	CU-15	WS /ws/chat	—	Pendiente
REQ-F-016	HU-16	CU-16	POST /api/pedidos/{id}/briefing	—	Pendiente
REQ-F-017	HU-17	CU-17	POST /api/legal/contratos	—	Implementado
REQ-F-018	HU-18	CU-18	POST /api/legal/contratos/{id}/firmar	—	Implementado
REQ-F-019	HU-19	CU-19	PATCH /api/pedidos/{id}/etapa	—	Implementado
REQ-F-020	HU-20	CU-20	POST /api/legal/paypal/webhook	—	Implementado
REQ-F-021	HU-21	CU-21	POST /api/legal/entregables/{id}/aprobar	—	Implementado
REQ-F-022	HU-22	CU-22	POST ¹⁷ /api/pedidos/{id}/tickets-revision	—	Implementado
REQ-F-023	HU-23	CU-23	POST /api/social/...	—	Pendiente

5.2. Estado global de cumplimiento

Estado	Cantidad	Porcentaje
Implementado	20	54 %
Pendiente	17	46 %
Total de requisitos trazados	37	100 %

Cuadro 4: Estado global de cumplimiento

Los módulos con mayor cierre son seguridad, perfil, catálogo y legal/pedido. Los módulos de comunicación en tiempo real (chat vía WebSocket) y social (seguimiento, comentarios, sorteos) concentran la mayoría de los requisitos pendientes, coherente con el desglose de cobertura de pruebas de la sección 8.D: precisamente esos módulos carecen de controlador implementado y, por tanto, de prueba automatizada asociada.

6. Protocolo experimental

Todas las mediciones reportadas en este documento fueron ejecutadas el 30 de julio de 2026 contra el sistema levantado con `docker compose up -d --build`, y sus datos crudos están versionados en `docs/mediciones/`. A continuación se describe el protocolo seguido para cada dimensión evaluada.

6.1. Rendimiento

- **Herramienta:** k6 v2.1.0.
- **Endpoint medido:** `GET /api/v1/catalogo?page=0&size=20`, con 202 servicios sembrados.
- **Configuración:** 50 usuarios virtuales (VUs), 30 segundos por corrida, 3 corridas independientes por escenario (caliente y frío), con aproximadamente 60 segundos de espera entre corridas.
- **Escenario frío:** `redis-cli FLUSHALL` inmediatamente antes de cada corrida, sin calentamiento previo.

6.2. Seguridad

- **Herramienta:** `curl`, contrastado contra el listado de riesgos OWASP Top 10.
- **Controles evaluados:** A01 (control de acceso roto), A02 (fallas criptográficas), A03 (inyección), A05 (configuración incorrecta de seguridad), A07 (fallas de identificación y autenticación) y A09 (fallas de registro y monitoreo).
- **Controles fuera de alcance en esta auditoría:** A04, A06, A08 y A10, por no contar con herramienta o escenario de prueba definido en la guía de esta entrega.

6.3. Cobertura de código

- **Herramienta:** JaCoCo 0.8.13 (plugin agregado en esta entrega).
- **Comando:** `./mvnw.cmd -B clean test`, sobre la suite JUnit 5 completa (18 clases de prueba, 89 pruebas).

6.4. Accesibilidad y calidad web

- **Herramienta:** Lighthouse 12.6.1 vía `@lhci/cli 0.15.1`.
- **Configuración:** `formFactor mobile`, throttling simulado (1638.4 Kbps / 150 ms RTT / desaceleración de CPU 4×), contra el contenedor de producción servido por Nginx en el puerto 4200 (no contra `ng serve`).
- Se ejecutaron dos corridas: una línea base (".antes") y otra tras aplicar un plan de mejora de cuatro fases ("después"), ambas en la misma sesión del 30 de julio de 2026.

6.5. Usabilidad (SUS)

- **Instrumento:** System Usability Scale (10 ítems, escala 1–5).
- **Tarea guiada prevista:** registro/inicio de sesión → explorar catálogo → crear pedido → revisar seguimiento → cerrar sesión.
- **Estado:** el instrumento, las instrucciones y la plantilla de consentimiento informado están listos en `docs/mediciones/sus/`, pero las sesiones con participantes reales (mínimo 10, externos al equipo) aún no se han ejecutado.

7. Resultados

7.1. Rendimiento

Se ejecutaron tres corridas de 30 segundos con 50 usuarios virtuales para cada uno de los dos escenarios definidos (caché caliente y caché fría), sobre el endpoint de catálogo. Los resultados agregados fueron los siguientes:

Métrica	Caché caliente	Caché fría	Umbral
n total (3 corridas)	4500	4500	—
Media	21.38 ms	13.62 ms	—
Mediana (p50)	13.01 ms	9.04 ms	—
p90	31.75 ms	22.39 ms	—
p95	50.17 ms	39.14 ms	¡200 ms / ¡500 ms
p99	205.60 ms	89.98 ms	—
Tasa de error 500	0.00 %	0.00 %	0 %
Throughput	48.5–49.0 req/s por corrida	—	—

Cuadro 5: Resultados de rendimiento

Ambos escenarios cumplen su umbral de p95. Se documenta de forma explícita una advertencia metodológica: el escenario "frío" mostró una latencia ligeramente menor que el "caliente", resultado sin explicación causal razonable dado que Redis no puede ser más lento que PostgreSQL para esta consulta. La causa identificada es que el script de k6 golpea siempre la misma combinación exacta de parámetros de paginación, por lo que la clave de caché (@Cacheable) es idéntica en ambos escenarios y, en la práctica, como mucho una de las 1500 peticiones por corrida es un miss real contra la base de datos. La diferencia observada (21.38 ms frente a 13.62 ms de media) se atribuye a ruido de ejecución (variabilidad de JIT/GC de la JVM y carga del contenedor), no a un efecto real de caché frío frente a caliente. El funcionamiento correcto del mecanismo de caché sí se verificó de forma aislada con curl y redis-cli: tras un FLUSHALL, la primera petición crea la clave en Redis con TTL de aproximadamente 60 segundos, y la invalidación (@CacheEvict) al crear, editar o eliminar un servicio es inmediata.

7.2. Seguridad

Se evaluaron seis controles del OWASP Top 10 contra el sistema levantado con Docker Compose (con el overlay docker-compose.medicion.yml para el control A02). Los seis controles evaluados cumplieron:

Control	Descripción	Resultado observado	Cumple
A01	Control de acceso roto	Un usuario que solicita el pedido de otro usuario recibe 403 Forbidden.	Sí
A02	Fallas criptográficas	Conector HTTPS adicional (puerto 8443) negocia TLSv1.3 con cifrado AEAD (TLS_AES_256_GCM_SHA384).	Sí
A03	Inyección	Consulta con carga tipo SQL injection en el filtro de catálogo devuelve 200 con lista vacía; consultas parametrizadas vía JPA Specification.	Sí
A05	Configuración incorrecta de seguridad	Cabeceras X-Content-Type-Options, X-Frame-Options, Content-Security-Policy, Referrer-Policy y Permissions-Policy presentes; HSTS confirmado sobre el conector TLS.	Sí
A07	Fallas de identificación y autenticación	Límite de tasa de login: intentos 1–5 devuelven 401, el intento 6 devuelve 429 con cabecera Retry-After.	Sí
A09	Fallas de registro y monitoreo	Registro estructurado de eventos de login exitoso y fallido, con correo, IP y, en éxito, identificador de usuario.	Sí

Cuadro 6: Resultados de seguridad OWASP

Quedan fuera del alcance de esta auditoría los controles A04 (diseño inseguro), A06 (componentes vulnerables o desactualizados), A08 (fallas de integridad de software y datos) y A10 (SSRF), por no contar con herramienta o escenario de prueba definido para ellos en la guía de la Tercera Entrega; se documentan como trabajo futuro.

7.3. Usabilidad

A la fecha de este informe no se cuenta con datos reales de la encuesta System Usability Scale: el instrumento, las instrucciones para el facilitador y la plantilla de consentimiento informado están listos en `docs/mediciones/sus/`, pero las sesiones con al menos 10 participantes externos al equipo aún no se han ejecutado. En cumplimiento del principio de no

maquillar resultados que rige el resto de este informe, no se completan aquí valores de ejemplo para media SUS, desviación típica o interpretación de la escala de Bangor. Esta sección debe completarse una vez recogidos los datos, siguiendo el procedimiento ya documentado.

7.4. Cobertura

La suite de pruebas unitarias del backend (18 clases, 89 pruebas, 0 fallos) se ejecutó con JaCoCo 0.8.13 sobre el comando `./mvnw.cmd -B clean test`:

Métrica	Cobertura obtenida	Umbral de referencia
Líneas	666 / 2893 = 23.0 %	60 % (no alcanzado)
Ramas (branches)	110 / 796 = 13.8 %	—
Complejidad	175 / 1039 = 16.8 %	—

Cuadro 7: Resultados de cobertura de código

Los módulos pedido, legal y catálogo no cuentan con pruebas unitarias de servicio; únicamente los módulos de seguridad, comunicación y social las tienen, lo que concentra en aquellos la mayor parte de las líneas sin cubrir. El umbral de referencia de la guía (60 %) no se cumple en esta entrega; se reporta el número real medido sin ajustar el umbral ni excluir paquetes para mejorar artificialmente el resultado. Queda como trabajo pendiente ampliar la cobertura de pruebas unitarias en los tres módulos señalados.

7.5. Accesibilidad

Se ejecutó Lighthouse (vía lhci) contra el contenedor de producción del frontend, antes y después de aplicar un plan de mejora de cuatro fases:

Categoría	Antes	Después	Umbral	Cumple
Performance	56/100	92/100	80	Sí
Accessibility	100/100	100/100	90	Sí
Best Practices	100/100	100/100	90	Sí
SEO	82/100	100/100	90	Sí

Cuadro 8: Resultados de Lighthouse

La causa raíz del bajo puntaje inicial de rendimiento fue la carga completa de la fuente Material Symbols (aproximadamente 3700 iconos, 1.1 MB) cuando la aplicación usa solo 44; ese recurso representaba el 71 % del peso total de la página. El subseteo de esa fuente y el auto-hospedaje de las tres fuentes utilizadas (Material Symbols, Inter, Manrope), junto con la activación de compresión gzip en Nginx, redujo el peso total transferido de 1554 KiB a 200 KiB (87 %) y bajó el First Contentful Paint de 9.5 s a 2.1 s, y el Largest Contentful Paint de

9.9 s a 2.8 s. Tras la mejora, la ejecución `lhci autorun` finalizó con las cuatro categorías por encima de su umbral, sin necesidad de modificar la configuración de umbrales del proyecto.

8. Bitácora de observaciones y cierre de brechas

Con el objetivo de sostener con evidencia el progreso declarado en el resumen ejecutivo, el equipo mantiene una bitácora re-verificada contra el código del repositorio (no por memoria ni por lo que declaraba una versión anterior del documento) en `docs/observaciones/OBSERVACIONES.md`. Cada observación resuelta indica el archivo, línea o comando con el que se comprobó, y el commit en el que la resolución aparece por primera vez.

8.1. Resumen por entrega

Entrega	Nota obtenida	Observaciones	Resueltas	Parciales	Pendientes
1A (09-06-2026)	9.5/10	5	2	0	3
1B (29-06-2026)	2.2/10	7	5	1	1
Auto-detectadas (revisión técnica)	—	3	2	0	1
Total		15	9 (60 %)	1 (7 %)	5 (33 %)

Cuadro 9: Resumen de observaciones por entrega

8.2. Detalle de observaciones

Código	Fuente	Criterio	Estado
OBS-01	Entrega 1A	Accesibilidad del repositorio	Resuelta — repositorio público y verificado por el docente
OBS-02	Entrega 1A	Coherencia técnica (versiones de la pila)	Pendiente y agravada — hoy existen 5 versiones distintas declaradas para la misma pila entre README, ADR y documentación C4
OBS-03	Entrega 1A	Coherencia técnica (conteo de tablas)	Resuelta — ADR-003 ya no cita una cifra exacta incorrecta

OBS-04	Entrega 1A	Cifras de carga sin fuente en el ADR	Pendiente — bajo esfuerzo, no bloqueante
OBS-05	Entrega 1A	Wireframe genérico	Pendiente — reemplazar por wireframe propio del dominio
OBS-06	Entrega 1B	Módulo de autenticación JWT ausente	Resuelta — AuthController, SecurityConfig, JwtService, 2FA y recuperación verificados en ejecución
OBS-07	Entrega 1B	Capa CRUD ausente	Resuelta — 24 controladores con servicios, repositorios y DTOs
OBS-08	Entrega 1B	Revocación Redis, refresh y controles OWASP ausentes	Resuelta — revocación en Redis, refresh vía cookie HttpOnly y 6 controles OWASP verificados
OBS-09	Entrega 1B	Pruebas JUnit 5 y Postman insuficientes	Parcialmente resuelta — 18 clases y 89 pruebas en verde; falta colección Postman de 20+ peticiones y cobertura 60%
OBS-10	Entrega 1B	Métricas de rendimiento p95 ausentes	Resuelta — 3 corridas en caliente y 3 en frío con k6, con reporte completo
OBS-11	Entrega 1B	Informe técnico como plantilla sin completar	Pendiente — subsanada mediante la redacción de este informe

OBS-12	Entrega 1B	Repositorio ejecutable y docker-compose	Resuelta — stack completo levanta healthy desde volumen limpio, con Makefile y digests
OBS-AUTO-01	Revisión técnica interna	Claims JWT incompletos	Resuelta — JwtService emite los 7 claims estándar del RFC 7519
OBS-AUTO-02	Revisión técnica interna	Procedimientos almacenados ausentes	Pendiente — brecha de mayor peso; 0 resultados para @Procedure/-NamedStored-ProcedureQuery en el backend
OBS-AUTO-03	Revisión técnica interna	Dockerfile de frontend faltante	Resuelta — artisync/Frontend/Dockerfile existe y el stack levanta completo

Cuadro 10: Detalle de observaciones

8.3. Etiquetado de versiones

Las etiquetas de control de versiones exigidas están publicadas en el repositorio remoto:

Etiqueta	Hito
v0.7.0	Entrega 1B — snapshot revisado por el docente el 29-06-2026
v0.7.1	Cierre parcial de la aplicación de observaciones de las Entregas 1A y 1B
v0.9.0-rc	Tercera Entrega — release candidate (versión de este informe)

Cuadro 11: Etiquetado de versiones

El equipo documenta deliberadamente que v0.7.1 marca el estado real alcanzado en el cierre

de observaciones —no un cierre al 100 %—, de modo que la etiqueta no afirme más de lo que el repositorio sostiene.

9. Amenazas a la validez

Con el mismo criterio de transparencia aplicado al resto del informe, se documentan a continuación las limitaciones conocidas del trabajo, tanto metodológicas como de alcance.

- El sistema fue probado en un único equipo de desarrollo (contenedores Docker locales); no se ha validado su comportamiento bajo condiciones de infraestructura de producción real ni con balanceo de carga entre múltiples réplicas.
- No se realizaron pruebas de carga con cientos o miles de usuarios reales concurrentes; las mediciones de rendimiento se limitan a 50 usuarios virtuales simulados con k6.
- La encuesta de usabilidad SUS no cuenta todavía con participantes; el instrumento está listo pero sin datos recogidos, por lo que no existe evidencia empírica de usabilidad percibida en esta entrega.
- Las mediciones de rendimiento pueden variar según el hardware del entorno de ejecución (se observó ruido de JIT/GC de la JVM entre corridas), por lo que los valores absolutos no deben generalizarse sin volver a medir en el entorno de destino.
- El diseño actual del script de carga de k6 no aísla de forma representativa el efecto del caché frío frente al caliente, ya que todas las peticiones de una corrida comparten la misma clave de caché; se documenta la limitación y se propone como trabajo futuro variar los parámetros de paginación por iteración.
- La cobertura de pruebas unitarias (23.0 % de líneas) no alcanza el umbral de referencia de la guía (60 %); tres de los siete módulos funcionales (pedido, legal, catálogo) carecen de pruebas de servicio.
- La auditoría de seguridad OWASP cubrió 6 de los 10 riesgos del Top 10 (A01, A02, A03, A05, A07, A09); los controles A04, A06, A08 y A10 no se evaluaron por falta de herramienta o escenario definido.
- La estrategia híbrida de acceso a datos (ADR-006) está aceptada pero no implementada: a la fecha de esta entrega, el 100 % del acceso a datos se resuelve vía ORM (JPA/Hibernate), sin procedimientos almacenados para las operaciones de joins múltiples o agregaciones que la guía exige encapsular; se verificó como atenuante que todas esas consultas usan parámetros nombrados, sin concatenación de entrada de usuario.
- Persisten inconsistencias documentales menores sobre las versiones exactas de la pila tecnológica entre distintos archivos del repositorio (por ejemplo, versiones de Angular y Spring Boot declaradas de forma distinta en README, ADR-001 y la documentación C4); los archivos de build (package.json, pom.xml) son la fuente de verdad y deben tomarse como referencia.
- 17 de los 37 requisitos trazados (46 %) permanecen sin implementar, concentrados en los módulos de comunicación en tiempo real (chat) y funciones sociales avanzadas (seguimiento, comentarios, sorteos).

10. Contribuciones de cada integrante (CRediT)

El equipo utiliza la taxonomía CRediT (Contributor Roles Taxonomy) para declarar de forma transparente el aporte de cada integrante, según lo registrado en `CONTRIBUTORS.md`.

Rol CRediT	Integrantes asignados
Conceptualización	Bone N., Carvajal J., Figueroa B., Ríos J.
Metodología	Bone N., Carvajal J., Figueroa B., Ríos J.
Software	Bone N., Carvajal J., Figueroa B., Ríos J.
Validación	Bone N., Carvajal J., Figueroa B., Ríos J.
Análisis formal	Figueroa B., Ríos J.
Investigación	Bone N., Carvajal J., Figueroa B., Ríos J.
Gestión de datos	Ríos J.
Redacción — borrador original	Bone N., Carvajal J., Figueroa B., Ríos J.
Redacción — revisión y edición	Bone N., Carvajal J., Figueroa B., Ríos J.
Visualización	Bone N., Carvajal J.
Supervisión	—
Administración del proyecto	Bone N., Carvajal J., Figueroa B., Ríos J.

Cuadro 12: Contribuciones CRediT

El equipo señala explícitamente que la asignación de roles puede ajustarse antes de la Entrega Final para reflejar con mayor precisión las actividades específicas realizadas por cada integrante.

11. Declaración ética

- No se utilizaron datos sensibles de terceros en ninguna fase del desarrollo o las pruebas del sistema.
- Los datos de prueba empleados en las mediciones de rendimiento y funcionalidad (catálogo sembrado con 202 servicios, usuarios de prueba) son ficticios, generados por el propio equipo para fines de evaluación.
- Para la prueba de usabilidad (SUS), pendiente de ejecución, se preparó una plantilla de consentimiento informado (`docs/etica/consentimientos/plantilla.md`) que explica al participante la tarea guiada, los datos que se registran (únicamente sus respuestas al cuestionario, asociadas a un código anónimo P01, P02, etc., nunca a su nombre), la naturaleza voluntaria de la participación y su independencia de cualquier calificación académica.
- El instrumento de consentimiento aclara expresamente que no se graba audio ni video durante las sesiones, y que las firmas de consentimiento se archivan fuera del control de versiones del repositorio, nunca junto con los datos crudos publicados.
- Se respeta la privacidad de cualquier participante futuro: los identificadores públicos en `sus-raw.csv` son códigos anónimos, no nombres ni correos electrónicos.

12. Referencias bibliográficas

1. ISO/IEC/IEEE 29148:2018, Systems and software engineering — Life cycle processes — Requirements engineering.
2. OWASP Foundation, "OWASP Top 10:2021," [En línea]. Disponible en: <https://owasp.org/Top10/>.
3. VMware Tanzu / Broadcom, "Spring Boot Reference Documentation," [En línea]. Disponible en: <https://docs.spring.io/spring-boot/>.
4. Docker Inc., "Docker Compose documentation," [En línea]. Disponible en: <https://docs.docker.com/compose/>.
5. PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," [En línea]. Disponible en: <https://www.postgresql.org/docs/16/>.
6. Google, "Angular Documentation," [En línea]. Disponible en: <https://angular.dev/>.
7. M. Jones, J. Bradley, N. Sakimura, RFC 7519 — JSON Web Token (JWT), IETF, 2015.
8. M. Nottingham, E. Wilde, RFC 7807 — Problem Details for HTTP APIs, IETF, 2016.
9. Google, "Lighthouse — Automated auditing for web quality," [En línea]. Disponible en: <https://developer.chrome.com/docs/lighthouse/>.
10. EcEmma / JaCoCo team, "JaCoCo Java Code Coverage Library," [En línea]. Disponible en: <https://www.jacoco.org/jacoco/>.
11. Grafana Labs, "k6 Documentation," [En línea]. Disponible en: <https://k6.io/docs/>.
12. S. Brown, "The C4 model for visualising software architecture," [En línea]. Disponible en: <https://c4model.com/>.
13. J. Brooke, "SUS: A quick and dirty usability scale," Usability Evaluation in Industry, 1996.
14. NISO / CASRAI, CRediT — Contributor Roles Taxonomy," [En línea]. Disponible en: <https://credit.niso.org/>.
15. PayPal, "PayPal Developer — Orders API v2," [En línea]. Disponible en: <https://developer.paypal.com/docs/api/orders/v2/>.
16. Redis Ltd., Redis Documentation," [En línea]. Disponible en: <https://redis.io/docs/>.
17. Flyway (Redgate), "Flyway Documentation," [En línea]. Disponible en: <https://documentation.red-gate.com/fd>.